SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4316

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Data Interpretation

Weightage: 20%

QUESTIONS: **Total Marks: 20**

Code of Conduct:

I P.NIKITHA/192324226 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

P. Nikitha
Signature of the student:

1: HTTP Response Analysis

A web server returns the following HTTP response header:
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with Content-Length and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

1. Identify any incorrect or invalid fields in the header.

The given HTTP response header is:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

The following issues are identified:

- Date: The weekday is incorrect. April 1, 2026, is a Wednesday, not a Monday.
- Content-Length: The value `-150` is invalid because the content length cannot be negative.
- Status code: `200 OK` indicates success, but it does not guarantee that every response header is valid.

The `Content-Type` and `Connection` fields are valid in this context.

2. Analyze the issue with Content-Length and explain its impact.

The `Content-Length` header specifies the size of the response body in bytes.

Given:

Content-Length: -150

Problems:

1. A negative content length is invalid under HTTP specifications.

2. The browser or client may reject the response or report a protocol error.
3. The client cannot reliably determine the expected response body length from this field.
4. Incorrect message framing can cause communication failures.

Correction: Specify the actual non-negative number of bytes in the response body, such as `150`, only if the body is exactly 150 bytes long.

3. Determine whether the status code matches the response correctness.

The status code is:

HTTP/1.1 200 OK

- `200 OK` means the request was successfully processed.
- However, the invalid `Content-Length` makes the response header malformed.
- Therefore, the status code alone does not establish that the entire response is valid.

Conclusion: The status code may describe a successful request, but the response still contains a protocol error that must be corrected.

4. Suggest corrections for the identified errors.

Corrected example:

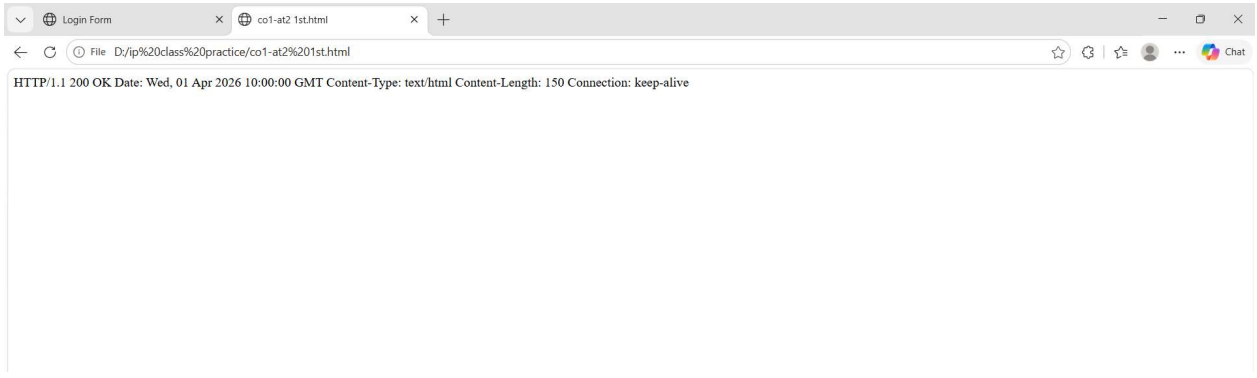
HTTP/1.1 200 OK

Date: Wed, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive



2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

1. Identify missing or improperly closed elements.

The given DOM structure contains the following issues:

- The `<p>` element is missing its closing tag `</p>`.
- The document is missing the `<!DOCTYPE html>` declaration.
- The `<html>` element is missing the `lang="en"` attribute.
- The `<head>` element is missing the character encoding declaration `<meta charset="UTF-8">`.

2. Analyze how the browser will interpret this structure.

1. The browser parses the HTML and constructs the Document Object Model (DOM).
2. When it encounters the `<div>` closing tag, the browser's HTML parser automatically closes the open `<p>` element.
3. The browser generally displays the page instead of reporting a fatal error.
4. Without `<!DOCTYPE html>`, browsers may use quirks mode, which can affect layout calculations.

3. Determine the impact on rendering and layout.

- Unexpected layout: Missing closing tags can cause unintended element nesting in some HTML structures.
- Spacing differences: Paragraph margins may affect the vertical spacing.
- Browser compatibility: Invalid markup can make debugging and consistent rendering more difficult.
- Accessibility: Incorrect document structure can reduce clarity for assistive technologies.
- Layout mode: Missing the DOCTYPE declaration may cause quirks-mode rendering.

4. Corrected code:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Test Page</title>

</head>

<body>

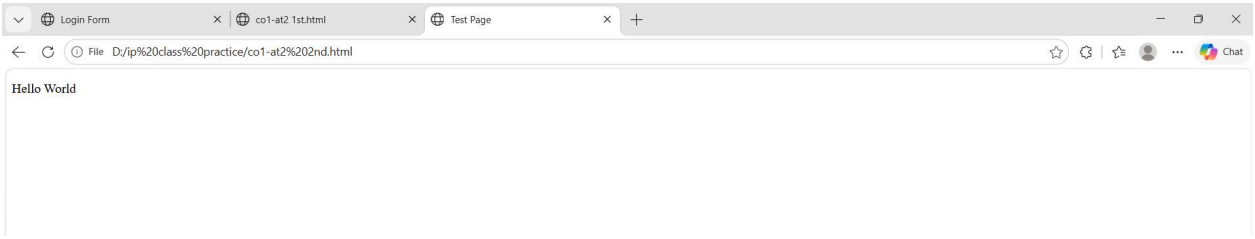
    <div>

        <p>Hello World</p>

    </div>

</body>

</html>
```



3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

1. Interpret the differences in data transmission between GET and POST.

Feature	GET	POST
Data transmission	Data is appended to the URL as query parameters	Data is sent in the request body
Visibility	Data appears in the URL	Data is not normally displayed in the URL
Data storage	May be saved in browser history and logs	Request body is not normally saved in browser history
Purpose	Retrieving information	Submitting data to the server
Security	Unsuitable for passwords	More suitable for submitting credentials over HTTPS

Example of GET:

GET /login?user=admin&pass=1234 HTTP/1.1

Host: example.com

Example of POST:

POST /login HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

user=admin&pass=1234

2. Analyze which method is more suitable for login and why.

POST is more suitable for login authentication.

Reasons:

1. Credentials are sent in the request body rather than the URL.
2. Passwords are less likely to be exposed through browser history or URL sharing.
3. POST is designed for submitting data to be processed by the server.
4. It avoids putting credentials in query parameters, which are commonly recorded in logs.

However, POST alone does not provide encryption. HTTPS is essential to protect credentials during transmission.

3. Identify security risks in using GET for authentication.

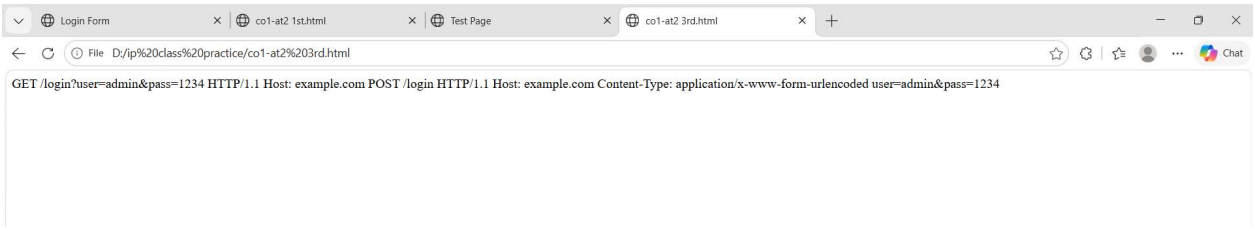
Using GET for login creates several security risks:

- URL exposure: Usernames and passwords appear in the address bar.
- Browser history: Credentials may remain in browsing history.
- Server logs: URLs may be recorded in web server and proxy logs.
- Shared links: A copied URL may expose credentials.
- Referrer leakage: URLs may be disclosed through referrer information in some circumstances.
- Caching: Sensitive URLs may be stored by caches or other systems.

Therefore, GET should not be used to transmit passwords during authentication.

4. Suggest best practices for secure login implementation.

1. Use the POST method for submitting login credentials.
2. Use HTTPS/TLS to encrypt communication between the client and server.
3. Store passwords using secure password-hashing algorithms such as Argon2id or bcrypt.
4. Validate credentials on the server side.
5. Use secure session cookies with `Secure`, `HttpOnly`, and appropriate `SameSite` attributes.
6. Apply rate limiting to reduce brute-force login attempts.
7. Use multi-factor authentication (MFA) where appropriate.
8. Avoid storing passwords in URLs, browser storage, or application logs.
9. Display generic login errors to avoid revealing whether a username exists.
10. Implement session expiration and secure logout.



4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
```

<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>

Questions:

1. Identify missing closing tags.
2. Analyze issues with the <img> tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

1. Identify missing closing tags.

The given HTML snippet contains these issues:

- The `<h1>` element is missing its closing tag `</h1>`.
- The `<p>` element is missing its closing tag `</p>`.
- The `<html>` element is missing the `lang` attribute.
- The document is missing the character encoding declaration `<meta charset="UTF-8">` and viewport meta tag.

2. Analyze issues with the `<img>` tag.

Given code:

<img src="image.jpg">

Issues:

1. Missing alternative text: The alt attribute should describe the image for screen readers.
2. Image availability: If image.jpg is missing or the path is incorrect, the image will not display.
3. Missing dimensions: Adding width and height can reduce layout shifts when the image loads.

Corrected example:

<img src="image.jpg" alt="Sample image"

width="300" height="200">

3. Determine how browsers will handle these errors.

1. Browsers generally attempt to recover from missing closing tags.
2. The HTML parser usually closes an open `<h1>` when it encounters the `<p>` start tag.
3. The open `<p>` is automatically closed when the parser reaches the end of the body.
4. The image displays if the file exists at the specified path.
5. Missing `alt` text reduces accessibility.
6. Without a viewport declaration, the page may not display optimally on mobile devices.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Sample</title>

</head>

<body>

    <h1>Welcome</h1>

    <p>This is a paragraph.</p>

    <img src="image.jpg"

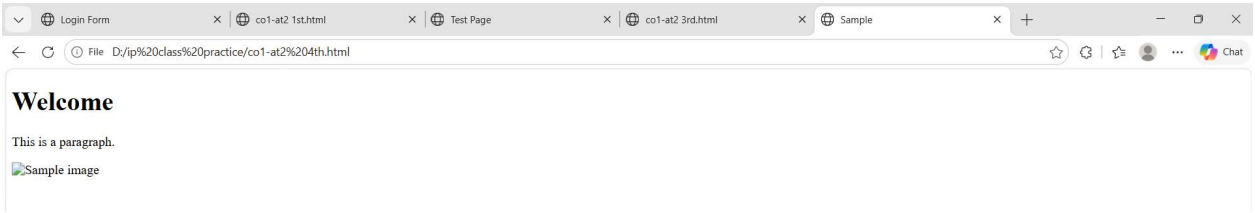
        alt="Sample image"

        width="300"

        height="200">

</body>

</html>
```



5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request

5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

1. Interpret each step in the communication flow.

Step 1: User enters a URL

The user enters a website address in the browser.

Step 2: DNS Lookup

DNS translates the domain name into an IP address.

Step 3: HTTP Request

The browser connects to the server and sends an HTTP request.

Step 4: Server Processing

The server processes the request and accesses data if needed.

Step 5: HTTP Response

The server returns a status code, headers, and response body.

Step 6: Browser Rendering

The browser processes HTML, CSS, and JavaScript to display the webpage.

2. Identify where delays can occur.

Delays may occur at different stages:

1. DNS lookup: The domain name takes time to resolve to an IP address.
2. Connection establishment: Establishing a TCP connection and performing a TLS handshake can add latency.
3. Network transmission: Slow internet connections or network congestion delay requests and responses.
4. Server processing: Complex computations or slow database queries increase response time.
5. Response transfer: Large HTML pages, images, and files take longer to download.

6. Browser rendering: Large CSS files, JavaScript execution, and complex page layouts delay display.

3. Analyze the role of DNS in the process.

DNS stands for Domain Name System. It translates human-readable domain names into IP addresses that computers use to communicate.

Example:

- Domain name: `www.example.com`
- DNS result: An IP address such as `93.184.216.34` (illustrative).

Functions of DNS:

- Resolves domain names to IP addresses.
- Helps the browser locate the appropriate server.
- Uses caching to reduce repeated lookups.
- Supports records such as A, AAAA, and CNAME.

Without successful DNS resolution, the browser generally cannot locate the server using that domain name.

4. Suggest optimizations to improve performance.

Area	Optimization
DNS	Use reliable DNS services and appropriate caching
Network	Use HTTP/2 or HTTP/3 where supported
Security	Reuse secure connections and enable TLS efficiently
Server	Optimize application code and database queries
Content	Compress responses using Brotli or gzip

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand